Ferramentas de Apoio - Semana 4

Extensões, ferramentas do terminal e do VS Code, documentação e leituras para Services/DI, Signals, Observables/RxJS e GitFlow

Desenvolvimento Front-End [Angular] T1 | Módulo 2 | Semana 4 | Este material reúne as ferramentas do terminal e do VS Code que facilitam o trabalho da semana, as extensões recomendadas, a documentação de referência (a "fonte da verdade" - não o palpite da IA) e leituras para reforçar cada tema fora do horário de aula. Nenhum material substitui a aula - use-os para revisar antes ou depois do encontro.

---

0. Antes de começar: versões do Node e do Angular

Item	Como conferir / o que fazer
Node.js	`node --version` . O **Angular 22** exige **Node ^22.22.3, ^24.15.0 ou ^26.0.0** (e TypeScript 6.0; fonte: angular.dev/reference/versions, conferida em 30/10/2026). Atualizar: `winget install OpenJS.NodeJS.LTS` (feche e reabra o terminal).
Angular CLI	`ng version` . Para criar o projeto da semana: `npm install -g @angular/cli` e `ng new biblioteca-web` . **Confira se o `ng version` mostra Angular 22** (os exercícios usam `@Service()` , componentes OnPush e zoneless por padrão).
Git	`git --version` . Se não existir: `winget install Git.Git` .
Sem poder instalar nada?	Use o **StackBlitz** (stackblitz.com): ambiente Angular no navegador, sem instalar Node. Serve para os Ex1-Ex3 de segunda e terça.

> **Dica**
> Se o PowerShell reclamar de `ng.ps1 ... running scripts is disabled` , use o **Prompt de Comando** ou o **Git Bash** no terminal do VS Code, ou rode uma vez `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` .

1. Ferramentas do terminal e do VS Code (já vêm instaladas)

Ferramenta	Onde fica	Para que serve nesta semana
Terminal integrado	`Ctrl + '`	Rodar `ng serve` , `ng g s` , `npm start` (playground RxJS) e os comandos Git
Source Control	`Ctrl + Shift + G`	Ver mudanças, fazer commits e **resolver conflitos** no editor de merge
Merge Editor	Clique no arquivo em conflito > *Resolve in Merge Editor*	Escolher Current/Incoming/Both lado a lado

DevTools > Application	`F12`	Conferir o `localStorage` salvo pelo `effect` (Ex3 de terça)
DevTools > Network	`F12`	Ver o `retry` do `HttpClient` fazendo 3 tentativas (Ex3 de quinta)
DevTools > Console	`F12`	Ver `LivroService criado` (singleton) e os logs do `effect`

2. Extensões do VS Code

Instale pela aba de Extensões (`Ctrl+Shift+X`), buscando pelo nome ou colando o identificador.

Extensão	Identificador	O que faz	Por que ajuda nesta semana
Angular Language Service	`Angular.ng-template`	Autocomplete e erros dentro dos templates HTML	Avisa se você esqueceu os `()` de um signal ou um import no componente
Error Lens	`usernamehw.errorlens`	Mostra o erro na própria linha	Acha o `NullInjectorError` /tipo errado sem abrir o terminal
GitLens	`eamodio.gitlens`	Autoria por linha, histórico e comparação de branches	Entender quem mudou o quê durante o GitFlow
Git Graph	`mhutchie.git-graph`	Desenha o grafo de branches e merges	Print obrigatório do Ex4 de quinta
GitHub Pull Requests	`GitHub.vscode-pull-request-github`	Criar e revisar PRs sem sair do VS Code	Revisão em dupla (Ex4)
Conventional Commits	`vivaxy.vscode-conventional-commits`	Formulário para montar `feat(escopo): ...`	Padroniza as mensagens de commit
Prettier	`esbenp.prettier-vscode`	Formata o código ao salvar	Mantém services e templates legíveis
Material Icon Theme	`PKief.material-icon-theme`	Ícones por tipo de arquivo	Reconhecer `.ts` , `.html` e pastas `services/` de relance

> **Dica**
> Você não precisa instalar todas. Se for escolher só duas para começar: **Angular Language Service** e **Git Graph**. Se algum identificador não aparecer na busca, procure pelo nome da extensão.

Extensão do navegador

Extensão	Onde	Para que serve

Angular DevTools	Chrome Web Store (busque por *Angular DevTools*)	Inspecionar a árvore de componentes, injetores e perfil de detecção de mudanças da aplicação rodando

3. Ferramentas e sites úteis (gratuitos)

- **Documentação do Angular** - angular.dev (inclui o *playground* no navegador).
- **RxJS** - rxjs.dev: referência de operadores e a árvore de decisão "qual operador usar".
- **RxMarbles** - rxmarbles.com: diagramas **interativos** de Observables (arraste as bolinhas e veja o resultado do `debounceTime`, `combineLatest`...).
- **StackBlitz** - stackblitz.com: Angular no navegador, sem instalar nada.
- **Learn Git Branching** - learngitbranching.js.org: tutorial interativo e visual de branches, merge e rebase.
- **Conventional Commits** - conventionalcommits.org: a especificação das mensagens de commit.
- **SemVer** - semver.org: versionamento semântico (as tags `v1.0.0`).
- **GitHub Docs** - docs.github.com: Pull Requests, revisão de código e proteção de branches.

4. Documentação de referência (a fonte da verdade)

Quando a IA e o colega discordarem, confira na documentação oficial. Se algum endereço mudar, busque pelo título no site:

Tema	Onde conferir
Injeção de dependência (visão geral)	angular.dev/guide/di
Criar e usar services	angular.dev/guide/di/creating-and-using-services
Hierarquia de injetores (escopo de providers)	angular.dev/guide/di/hierarchical-dependency-injection
Signals (`signal`, `computed`, `effect`)	angular.dev/guide/signals
`linkedSignal`	angular.dev/guide/signals/linked-signal
`input()` (propriedades de entrada)	angular.dev/guide/components/inputs
HttpClient	angular.dev/guide/http
RxJS (Observables, operadores)	rxjs.dev/guide/overview
Git (livro oficial, em português)	git-scm.com/book/pt-br/v2

> **Atenção às versões**
> O curso usa o **Angular 22**. Nele: `@Service()` é o jeito padrão de criar um service (atalho de `@Injectable({ providedIn: 'root' })`, que continua válido e é necessário para injeção por construtor e escopos não-raiz); componentes novos nascem **OnPush**; aplicações novas são **zoneless**. Vídeos e tutoriais antigos usam `@Injectable`, `app.component.ts`, `*.component.ts` e `zone.js`: o conceito é o mesmo, só mudam os nomes. Se algo da doc não existir no seu projeto, confira a versão em `ng version`.

5. Leituras de apoio (em português)

> **Importante**
> Leituras localizadas por busca na web em 30/10/2026 e conferidas **apenas pelo título e descrição exibidos na busca**; o professor ainda deve ler cada uma antes de indicar à turma.

- Services e injeção de dependência no Angular: o que são e como funcionam? (Alura)
- Entendendo os Signals do Angular (Alura)
- Entendendo BehaviorSubject e Observable no RxJS (Hora de Codar)
- Git Flow: o que é, quando utilizar e como funciona com passo a passo (Alura)
- Pull Request no Git (DataCamp)
- Fluxo de trabalho de uma ramificação de funções do Git (Atlassian)

6. Vídeo-aulas: como encontrar (sem indicar o que não foi conferido)

> **Importante**
> A busca automática feita para montar este material **não retornou vídeos do YouTube verificáveis**; por isso **nenhum link de vídeo foi inventado**. Antes de publicar no AVA, pesquise, assista ao trecho principal e só então inclua os que combinarem com o curso. Termos de busca sugeridos abaixo, por aula:

Aula	Termos de busca sugeridos (YouTube)
02/11 - Services e DI	`Angular service injeção de dependência inject()` · `Angular providedIn root singleton` · `Angular InjectionToken`
03/11 - Signals	`Angular signals computed effect` · `Angular input output model signals` · `Angular OnPush`
03/11 - Observables	`RxJS introdução Observable subscribe` · `RxJS operadores map filter pipe` · `Observable vs Promise`
05/11 - RxJS aplicado	`Angular BehaviorSubject async pipe` · `RxJS debounceTime switchMap busca` · `Angular HttpClient Observable`
05/11 - GitFlow	`GitFlow na prática` · `Pull Request code review GitHub` · `resolver conflito de merge VS Code`

> **Como usar este material**
> Leia ANTES da aula para chegar com uma primeira impressão, ou DEPOIS para reforçar um ponto. O conteúdo oficial e completo da semana está nos slides e nas pastas de exercícios.